

# 基于 RK3588 的端侧多场景人流量感知与异常行为 智能预警系统

## 摘要

本作品面向重点区域智能管理需求，设计并实现了一套基于 RK3588 边缘计算平台的实时视频监控与异常行为预警系统。系统以 MIPI 摄像头作为视频采集端，在边缘端完成图像采集、图像预处理、目标检测、多目标跟踪、规则分析、视频推流和数据上报等任务。核心算法采用 YOLO 目标检测模型完成人员识别，并通过 RKNN 模型转换与部署流程将模型运行在 RK3588 NPU 上，提升端侧推理效率；预处理阶段调用 RGA 硬件加速完成 NV12 到 RGB 的格式转换、图像缩放和输入尺寸适配；目标关联阶段引入 ByteTrack，使系统能够为同一人员分配稳定的 id，支撑跨帧轨迹分析。在应用层，系统结合 ROI 区域配置、目标位置、轨迹持续时间和运动状态，实现区域内人数统计、重点区域进入识别、长时间停留预警等功能。系统采用“视频流+结构化数据”分离的设计思路：视频画面经 GStreamer/RTS 推送至 MediaMTX，再由 WebRTC 或浏览器端播放；人数、FPS、温度、上报延迟和异常事件等数据通过 HTTP json 上报至 FastAPI 后台，并在 PC 端网页实时展示。测试结果表明，系统在 640×640 模型输入下可达到 30 FPS，RGA 预处理耗时约 2.4-2.9ms，NPU 推理与后处理耗时约 17-24 ms，具备较好的实时性、可扩展性和工程部署价值。

## 第一部分 作品概述

### 1.1 功能与特性

本作品实现了从视频采集、边缘智能分析到后台展示的完整闭环。系统可实时采集摄像头画面，检测画面中的人员目标，并通过 ByteTrack 为目标分配 track\_id，实现跨帧身份关联。在此基础上，系统支持 ROI 区域设置，可统计区域内实时人数，并对长时间停留、重点区域进入等异常状态进行识别和上报。后台页面可同步展示实时视频、人数、FPS、系统温度、上报延迟和事件记录，适用于需要现场快速响应的智能监控场景。

### 1.2 应用领域

本作品适用于智慧校园、智慧园区、办公楼宇、图书馆、实验室、通道出入口和设备机房等需要人员状态感知的区域。系统可用于人流量统计、限制区域闯入检测、异常停留提醒、现场状态可视化和安全管理辅助决策。与传统云端视频分析方案相比，本系统将主要计算任务部署在边缘端，可减少原始视频上传带来的带宽压力，提高现场响应速度，并降低对外部网络稳定性的依赖。

### 1.3 主要技术特点

系统采用端侧 AI 推理架构，利用 RK3588 内置 NPU 加 YOLO 模型推理，降低 CPU 负载并提升实时性。图像预处理阶段调用 RGA 完成格式转换与缩放，减少软件处理耗时。软件结构采用模块化设计，包含 camera、RGA preprocess、detector、post\_processor、tracker、event\_analysis、rtsp\_streamer、reporter 和 backend 等模块，便于调试、替换和扩展。后台采用 FastAPI 接收数据，MediaMTX 提供视频转发，使视频展示与数据统计互不干扰。

### 1.4 主要性能指标

| 指标   | 当前实现结果        | 说明               |
|------|---------------|------------------|
| 硬件平台 | RK3588 边缘计算平台 | 端侧采集、预处理、推理和事件分析 |

|          |             |                          |
|----------|-------------|--------------------------|
| 摄像头输入    | MIPI 摄像头    | 通过 V4L2 获取帧数据            |
| 图像预处理    | RGA 硬件加速    | NV12 转 RGB, 2.4-2.9ms    |
| 跟踪算法     | ByteTrack   | 输出 track_id 与目标轨迹        |
| 实时帧率     | 约 29-30 FPS | 满足实时视频分析要求               |
| 推理与后处理耗时 | 约 17-24 ms  | 随场景目标数略有变化               |
| 单帧总耗时    | 约 30-34 ms  | 采集到分析链路                  |
| 数据上报时延   | 80-100ms    | HTTP JSON ， FastAPI 后台接收 |

## 1.5 主要创新点

- 1、采用 RK3588 端侧闭环部署，实现采集、推理、分析和上报本地完成。
- 2、结合 YOLO 检测与 ByteTrack 跟踪，实现人员目标跨帧 ID 关联。
- 3、基于 ROI 与轨迹状态设计停留预警规则，提高场景适应性。
- 4、采用视频流与 JSON 数据分离的方式，兼顾实时画面与统计展示。
- 5、利用 RGA 与 NPU 协同加速，提高嵌入式平台实时处理能力。

## 1.6 设计流程

系统设计流程按照“需求分析-硬件搭建-采集调试-预处理加速-模型部署-后处理适配-目标跟踪-事件分析-推流上报-后台展示”的路径推进。该流程先保证底层视频链路稳定，再逐步接入 NPU 推理和应用层功能，便于分阶段测试和问题定位。

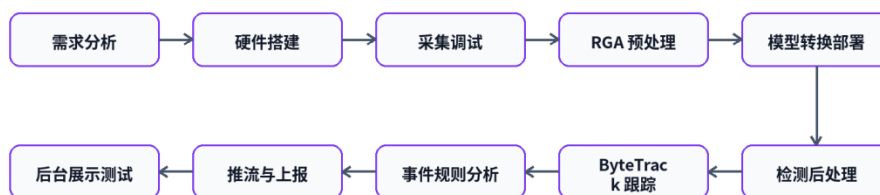


图 1 设计流程图

## 第二部分 系统组成及功能说明

---

### 2.1 整体介绍

本系统硬件部分以 RK3588 边缘计算平台为核心，围绕“视频采集-图像预处理-模型推理-结果输出-后台展示”的工作链路进行搭建。RK3588 作为边缘端主控单元，负责协调摄像头输入、图像处理、AI 推理、网络通信和调试管理等任务。系统前端接入 MIPI 摄像头模组，摄像头通过 CSI 接口向开发板输入 NV1 格式视频流，边缘端程序通过 V4L2 驱动接口完成摄像头设备打开、采集格式设置、缓冲区申请和图像帧读取，为后续图像处理提供原始视频数据。

在图像预处理阶段，系统调用 RK3588 板载 RGA 硬件加速模块，对摄像头采集到的 NV12 图像进行格式转换、尺寸缩放和输入适配处理，将原始视频帧转换为 YOLO v8 模型所需的 RGB 输入数据。相比单纯依赖 CPU 进行图像转换，RGA 能够有效降低 CPU 占用，使主处理器更多承担任务调度、逻辑判断和通信管理工作，提高整体实时性。

在推理阶段，系统将训练并转换后的 RKNN 模型部署到 RK3588 的 NPU 上运行。NPU 负责执行 YOLO 目标检测模型的前向推理，输出目标框、类别和置信度等信息。根据平台模型部署流程，模型在板端运行时一般需要经历模型初始化、输入数据前处理、NPU 推理、输出结果后处理和资源释放等步骤，系统在此基础上进一步加入 ByteTrack 多目标跟踪和 ROI 规则分析，实现人员检测、目标 ID 关联、区域人数统计和长时间停留判断等功能。

系统通信部分通过 Wi-Fi 与 PC 后台建立局域网连接。边缘端一方面将叠加检测框、目标 ID 和规则分析结果后的视频画面通过 RTSP 输出，供 PC 后台或浏览器端实时查看；另一方面将人数统计、FPS、设备状态、事件信息和时间戳等结构化数据封装为 JSON，通过 HTTP 接口上传至后台。PC 端主要承担视频查看、数据接收、状态展示、异常事件记录和调试结果保存等功能，从而形成“边缘端负责实时分析，后台端负责可视化展示和数据管理”的分工结构。

在调试与维护方面，系统保留了多种调试接口。开发过程中可通过 SSH 远程登录开发板，查看程序运行日志、模型推理耗时和网络连接状态；可通过串口观察系统启动信息和底层驱动输出。同时，程序运行过程中会输出摄像头采集、RGA 处理、RKNN 推理、视频推流和 HTTP 上报等关键环节的日志信息，便于定位摄像头无画面、模型加载失败、推流中断、后台接收异常等问题。整体硬件系统结构清晰，各模块分工明确，能够支撑实时视频采集、端侧 AI 推理、异常行为分析和后台可视化展示等功能需求。

## 2.2 硬件系统介绍

### 2.2.1 硬件整体介绍

本系统以 ELF2 开发板作为边缘计算核心，主控芯片为 Rockchip RK3588 系列处理器。该平台集成包括 4 核 Cortex-A55 和 4 核 Cortex-A76、Mali GPU、三核 NPU 以及 RGA 图像硬件加速单元，能够在端侧完成视频采集、图像预处理、目标检测、目标跟踪和事件分析等任务。开发板提供 HDMI、USB、千兆以太网、MIPI-CSI、调试串口、音频接口和扩展排针等外设接口，便于连接摄像头、显示设备和上位机后台。

在本项目中，ELF2 主控板主要承担四类功能：第一，通过 MIPI-CSI 接口接收摄像头采集的视频数据；第二，利用 RGA 完成 NV12 到 RGB、缩放和 letterbox 等图像预处理；第三，将处理后的图像送入 RKNN 模型，在 NPU 上完成 YOLO 人员检测；第四，通过 CPU 运行 ByteTrack 跟踪、ROI 区域判断、长时间停留分析和 JSON 数据上报逻辑。处理后的视频流通过网络接口推送至 PC 端后台，实现实时画面展示和统计数据可视化。

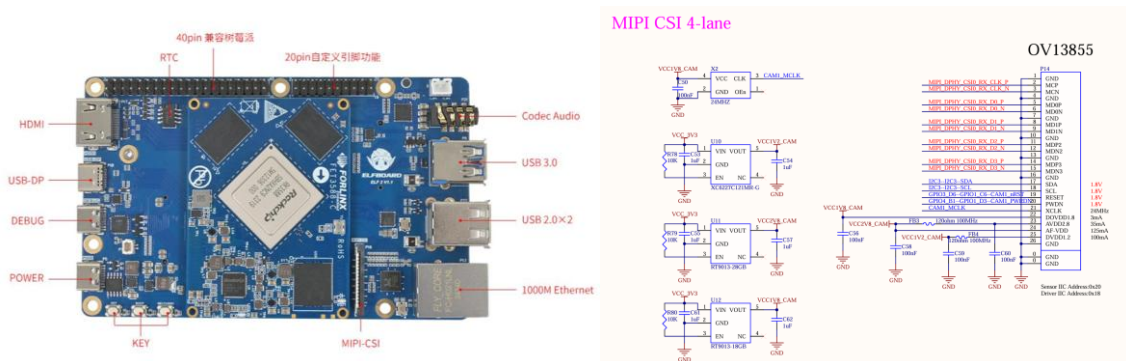


图 2 硬件电路图

## 2.3 软件系统介绍

### 2.3.1 软件整体介绍

软件系统采用模块化结构，边缘端程序主要由 C/C++ 实现，围绕摄像头采集、RGA 预处理、RKNN 推理、YOLO 后处理、ByteTrack 跟踪、ROI 事件分析、RTSP 推流和 HTTP 上报构建处理流水线。PC 后台采用 FastAPI 提供数据接口和网页服务，MediaMTX 负责视频流接入与浏览器播放适配。系统运行时，边缘端持续产生两类输出：一类是叠加检测框和事件信息的视频流，另一类是包含人数、FPS、温度、延迟和事件列表的 JSON 数据。

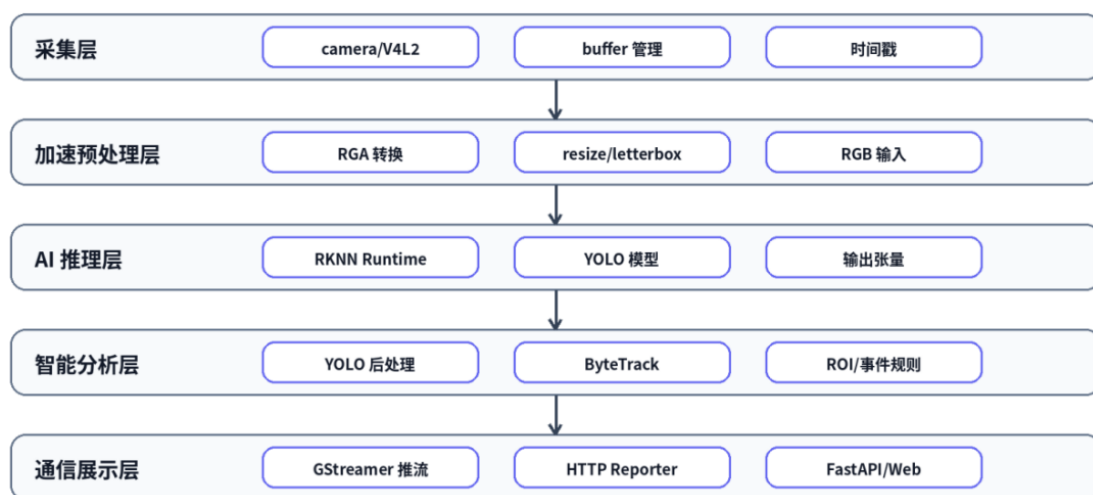


图 3 系统软件流程图

### 2.3.2 软件各模块介绍

#### (1) 顶层主流程模块

主程序负责完成系统配置读取、各子模块初始化、主循环调度、异常处理和资源释放。程序启动后，首先读取模型路径、摄像头参数、ROI 区域、推流地址及后台接口地址等配置项；随后完成 camera、RGA、RKNN、tracker、event\_analyzer、streamer 和 reporter 等模块的初始化，并进入逐帧处理循环。主循环中依次完成图像采集、预处理、模型推理、后处理、目标跟踪、事件分析、视频叠加、推流和上报，同时记录各环节耗时，用于计算系统实时帧率和评估瓶颈位置。

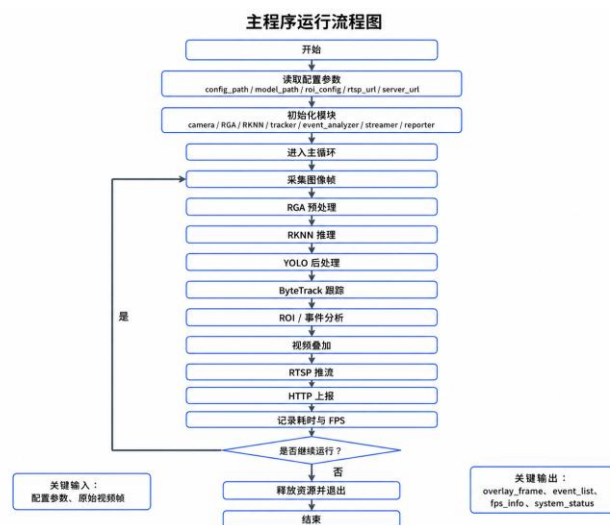


图 4 主程序运行流程图

## (2) 采集层：camera / V4L2 模块

采集层负责从 MIPI 摄像头获取原始视频帧。系统通过 V4L2 接口打开摄像头设备，设置采集分辨率、像素格式和帧率，并采用 MMAP 方式申请采集缓冲区。摄像头输出格式为 NV12，采集到的每帧数据均附带时间戳和缓冲区索引，用于后续推理、跟踪及时延统计。

其中，camera\_init()主要完成设备打开、格式设置、缓冲区申请、内存映射和采集启动；camera\_read\_frame()用于在运行阶段不断从驱动队列中取出图像帧，并将帧数据组织为 FrameBuffer 结构体返回给上层模块。

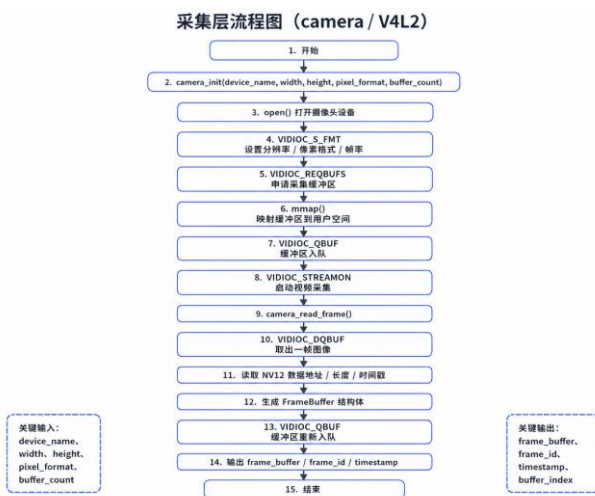


图 5 采集层流程图

### (3) 缓存管理模块

为避免在实时处理过程中频繁进行内存申请与释放，系统在采集层与预处理层之间设置了缓冲区管理模块。该模块统一维护摄像头采集缓冲区、预处理输出缓冲区以及推理输入缓冲区的使用状态，通过“申请-占用-释放”的方式完成固定缓存池管理，保证多帧连续处理过程中的数据完整性和访问效率。

`buffer_pool_init()` 用于创建固定大小的缓冲区池，并建立缓冲区编号与内存地址之间的映射关系；`buffer_get_free()` 用于获取当前可用缓冲区；`buffer_release()` 则在当前帧处理完成后释放对应缓冲区，防止缓存覆盖或内存泄漏。

| 函数/模块                                                   | 关键输入变量                                              | 关键输出变量                                                               |
|---------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------|
| <code>buffer_pool_init(buffer_count, frame_size)</code> | <code>buffer_count</code> 、 <code>frame_size</code> | <code>BufferPool</code> 、 <code>buffer_map</code>                    |
| <code>buffer_get_free()</code>                          | <code>BufferPool</code>                             | <code>buffer_handle</code> 、 <code>data_ptr</code> 、 <code>fd</code> |
| <code>buffer_release(buffer_index)</code>               | <code>buffer_index</code> 、 <code>BufferPool</code> | <code>state_update</code>                                            |

### (4) 加速预处理层与 AI 推理层

预处理层负责将摄像头采集得到的 NV12 图像转换为模型推理所需的输入格式。系统调用 RK358 板载 RGA 硬件模块完成颜色空间转换和尺寸缩放，将原始图像转换为 RGB 输入，并根据 YOLO 模型输入尺寸进行 `resize` 或 `letterbox` 处理。预处理过程中同时记录缩放比例和边缘填充值，以便在后处理阶段将检测框坐标还原到原始图像坐标系。

AI 推理层基于 RKNN Runtime 运行目标检测模型。`detector_init()` 负责加载 `.rknn` 模型文件并查询输入输出张量属性；`detector_run()` 将预处理后的 `input_tensor` 送入 NPU 执行推理，并获取原始输出张量；`post_process()` 则对 YOLO 模型输出进行解析、置信度筛选、NMS 去重和坐标还原，最终生成检测结果列表 `detections`。



图 6 预处理与 AI 推理流程图

## (5) 智能分析层：ByteTrack 跟踪与 ROI 规则模块

智能分析层在单帧检测结果的基础上完成跨帧目标关联和业务规则分析。系统首先将当前帧检测结果输入 ByteTrack 多目标跟踪器，由其基于目标位置、置信度和历史轨迹进行目标关联，并为每个目标分配稳定的 track\_id。随后，ROI 规则模块根据目标框中心点或脚点位置判断目标是否位于指定监控区域内，并进一步统计人数、计算停留时长和判定异常停留事件。

在该层中，tracker\_update() 负 Kalma 预测、检测框匹配和轨迹状态更新；event\_analyzer\_update() 则根据 ROI 配置和时间阈值生成统计结果与事件结果，是实现人员统计和停留识别功能的核心模块。

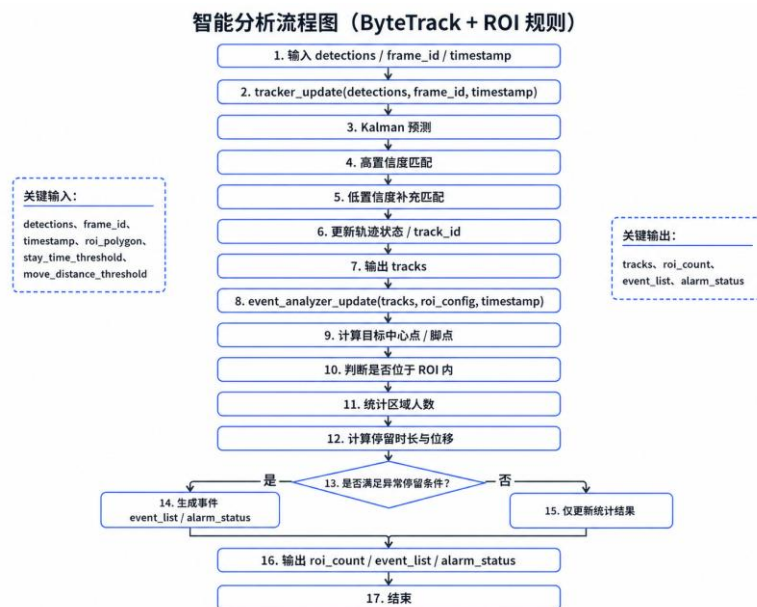


图 7 智能分析流程图（ByteTrack + ROI 规则）

## （6）通信展示层：视频推流、数据上报与后台展示

通信展示层用于将边缘端处理结果输出到后台。系统在原始画面上叠加检测框、轨迹编号、ROI 区域和统计信息后，通过 GStreamer 管线完成 H.264 编码和 RTSP 推流；与此同时，将人数统计、事件信息、设备状态和时间戳等结构化数据封装为 JSON 数据包，以 HTTP POST 方式上报 PC 后台。

系统后台界面采用 Web 可视化方式展示边缘端运行结果，整体页面以深色背景为主，顶部为系统标题栏，显示“边缘智能监控后台”及系统功能说明，包括 YOLO 检测、ByteTrack 跟踪、ROI 事件分析和实时视频展示等内容。页面右上角显示当前系统时间，便于与设备发送时间、后台接收时间进行对照。

整体来看，该软件界面将实时视频、性能指标、智能分析结果、系统状态和历史趋势集中在同一页面中展示，实现了边缘端检测结果到后台可视化的闭环。后台服务采用 FastAPI 接收边缘端上报的数据，并将最新状态写入缓存或数据库；Web 端则定时读取状态接口，并结合 MediaMTX 转发的视频流，实现实时画面、系统指标和异常事件的统一展示。通过推流与上报双通道并行工作，系统能够同时满足视频查看与业务数据展示的需求。



图 8 通信展示流程图

## 第三部分 完成情况及性能参数

### 3.3 整体介绍（整个系统实物的正面、斜 45°全局性照片）

本系统硬件部分以 RK3588 边缘计算开发板为核心，外接 OV13855 MIPI-CSI 摄像头模组，用于完成校园重点区域的视频采集、端侧推理和网络上传。整体硬件由主控计算模块、图像采集模块、网络通信模块、电源模块和调试显示模块组成。其中，OV13855 摄像头通过 MIPI CSI 4-lane 接口将原始视频数据传输至 RK3588 主控板；RK3588 内部集成 CPU、NPU、GPU 和 RGA 图像加速单元，分别承担系统调度、目标检测推理、图像处理和辅助显示等任务；网络接口用于将检测结果、异常事件和实时视频流上传至 PC 端后台。

在实际工作过程中，摄像头首先采集监控区域画面，并通过 V4L2 驱动送入系统缓存；随后 RK3588 使用 RGA 完成图像格式转换和尺寸缩放，再调用 NPU 运 YOLO 检测模型，完成行人目标识别；CPU 负责 ByteTrack 多目标跟踪、ROI 区域判断、长时间停留分析和 JSON 数据上报。处理后的视频流通过 RTSP/MediaMTX 传输到后台网页，实现实时画面、人数统计、设备状态和异常事件的统一展示。整体硬件结构紧凑，接口资源丰富，能够满足边缘端实时视频分析系统对采集、计算、通信和部署稳定性的要求。



图 9 系统全局图



图 10 现场测试图

### 3.2 工程成果

界面左侧为实时监控画面区域，用于显示由边缘端推送的视频流。该区域主要承载摄像头实时画面、检测框、目标 ID、ROI 区域及异常事件提示等可视化信息。截图中视频窗口显示 “Error: source of path 'live' has timed out, retrying in some seconds”，说明当前 WebRTC 视频源暂时未正常拉取。页面底部也给出提示：若画面无法显示，应先单独打开视频地址确认 MediaMTX 与板端 RTSP 是否正常。该提示有利于现场调试时快速区分是前端显示问题、流媒体服务问题，还是边缘端推流问题。

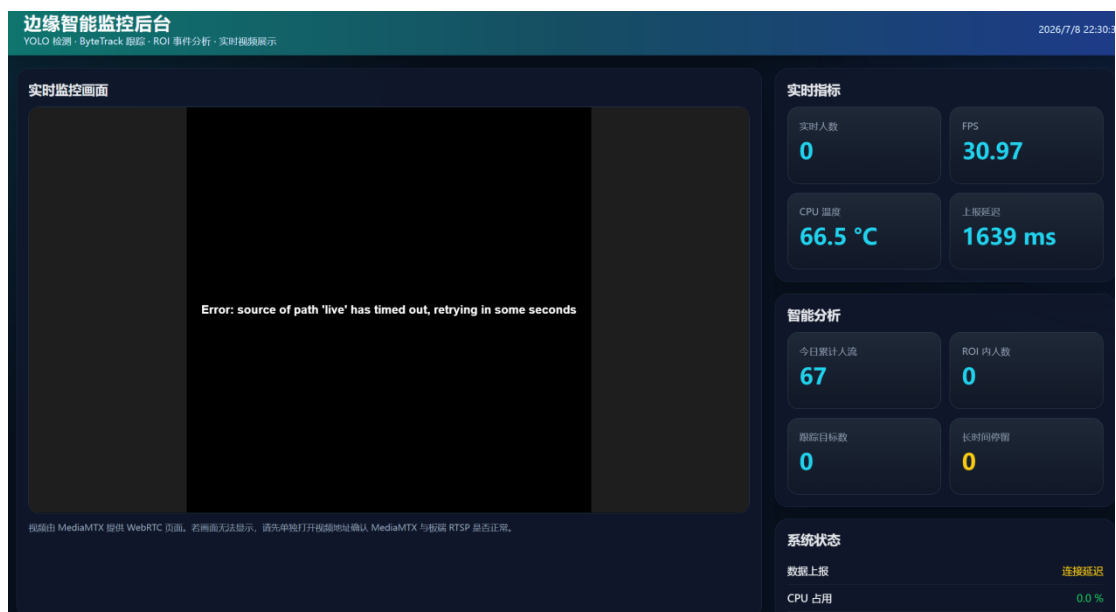


图 11 后台 UI 界面图

界面右侧上方为实时指标区域，用于展示系统当前运行状态。从下图可以看到，该区域包括实时人数、FPS、CPU 温度和上报延迟等关键指标。其中 FPS 显示为 30.97，说明系统当前仍保持接近 30 帧每秒的处理能力；CPU 温度为 66.5 °C，用于判断设备长时间运行时的散热状态；上报延迟为 1639 ms，用于反映边缘端数据发送到后台接收之间的时间差。这些指标能够帮助用户实时判断系统是否处于稳定运行状态。

右侧中部为智能分析区域，主要展示人员检测与 ROI 分析结果。页面中包含今日累计人流、ROI 内人数、跟踪目标数和长时间停留数量等信息。截图中今日累计人流为 67，说明后台已累计接收到人员统计数据；ROI 内人数、跟踪目标数和长时间停留均为 0，表示当前画面中暂未识别到处于 ROI 区域内的有效目标，也未触发停留类异常事件。该区域用于将算法分析结果以数字化方式呈现，便于管理人员快速了解现场状态。

系统状态区域用于展示数据上报、CPU 占用、设备发送时间和后台接收时间等运行信息。截图中数据上报状态显示为“连接延迟”，说明后台能够接收到数据，但通信存在一定延时；该模块可以用于判断边缘设备、后台服务和网络链路之间的同步状态。

页面下方还设置了人数趋势图、异常事件列表和跟踪目标列表。人数趋势图用于显示最近一段时间内的人流变化情况，能够直观看出人数变化的峰值和波动趋势。异常事件列表用于记录触发的事件类型、目标 ID、持续时间和所在区域；跟踪目标列表用于展示当前正在跟踪的目标信息，包括目标 ID、置信度、ROI 状态和停留状态。



图 12 后台数据图

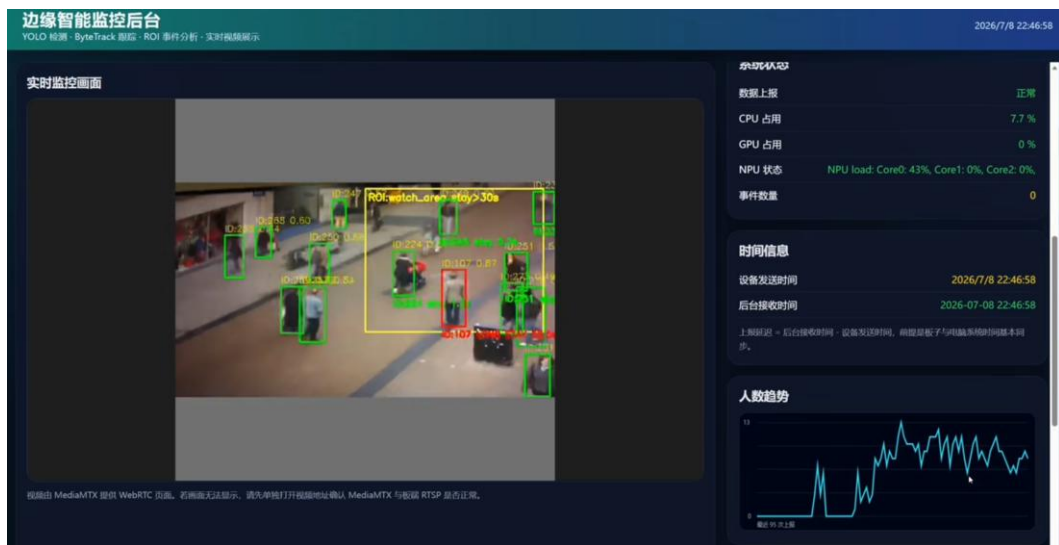


图 13 后台数据图

### 3.3 特性成果

| 功能项      | 完成情况 | 说明                          |
|----------|------|-----------------------------|
| 摄像头采集    | 已完成  | V4L2 采集 NV12 数据，支持连续取帧      |
| RGA 预处理  | 已完成  | NV12 转 RGB，并缩放至 640×640     |
| RKNN 推理  | 已完成  | YOLO 模型运行在 RK3588 NPU       |
| YOLO 后处理 | 已完成  | 支持多尺度输出解析、阈值过滤和 NMS         |
| 多目标跟踪    | 已完成  | ByteTrack 输出 track_id 和轨迹状态 |
| ROI 区域分析 | 已完成  | 支持重点区域内人数统计                 |
| 长时间停留识别  | 初步完成 | 根据 track_id、ROI 和停留时长触发事件   |
| 视频推流     | 已完成  | RTSP/MediaMTX/WebRTC 显示     |
| 数据上报     | 已完成  | HTTP JSON 上报状态和事件           |
| 后台可视化    | 已完成  | 显示实时指标、系统状态和事件信息            |
| 历史统计     | 可扩展  | 后续可接入数据库保存和检索               |

| 量化指标         | 当前结果         | 说明                 |
|--------------|--------------|--------------------|
| 模型输入尺寸       | 640×640×3    | RGB                |
| 实时帧率         | 约 29-30 FPS  | 实际结果随目标数量和后台网络状态波动 |
| RGA 预处理耗时    | 约 2.4-2.9 ms | 格式转换与缩放            |
| NPU 推理与后处理耗时 | 约 17-24 ms   | 包含模型推理和输出解析        |
| 单帧总耗时        | 约 30-34 ms   | 满足实时分析要求           |
| 视频传输方式       | RTSP/WebRTC  | 适合局域网实时查看          |
| 数据上报方式       | HTTP JSON    | 便于后台扩展数据库和事件检索     |

从测试结果看，系统在当前输入尺寸下能够达到接近实时视频帧率，RGA 与 NPU 的硬件加速明显减轻了 CPU 压力。后续如果接入多摄像头或更复杂行为识别规则，可通过多线程流水线、异步队列和模型轻量化进一步提升吞吐率。

## 第四部分 总结

---

### 4.1 可扩展之处

本系统后续可从多路接入、模型精度和事件分析三个方向继续扩展。首先，在硬件资源利用方面，可进一步发挥 RK3588 三核 NPU 的并行能力，通过多 RKNN Context 和多线程流水线，将采集、预处理、推理、跟踪、推流和上报解耦，提高多路视频接入能力和系统吞吐率。其次，在检测精度方面，可采集校园实际场景数据，对 YOLO 模型进行针对性微调，并引入更高分辨率输入或 ROI 二次检测机制，以提升远距离小目标、遮挡人员和复杂光照场景下的识别效果。最后，在智能分析方面，可扩展越界、逆行、异常聚集、摔倒检测等规则，并结合历史数据存储、事件回放和告警分级机制，形成更加完整的智慧校园边缘监控平台。

### 4.2 心得体会

本作品的研发过程覆盖了嵌入式 Linux、计算机视觉、硬件加速、网络通信和 Web 可视化等多个方向。项目初期首先完成摄像头采集链路调试，通过 V4L2 理解了驱动缓冲区申请、MMAP 映射、QBUF/DQBUF 队列管理和 STREAMON 启动采集的完整流程。这个阶段看似只是“把摄像头画面取出来”，但实际涉及设备节点、像素格式、分辨率、缓冲区长度和帧时间戳等细节，一旦某个参数不匹配，就可能导致画面异常或程序阻塞。

在图像预处理阶段，系统接入 RGA 进行 NV12 到 RGB 的格式转换和尺寸缩放，使我认识到嵌入式平台中 CPU、NPU 和专用硬件加速模块协同工作的重要性。相比直接使用 CPU 处理图像，RGA 能够显著降低预处理耗时，但它对内存对齐、stride、buffer 大小和格式描述要求更严格，因此调试时需要同时关注代码逻辑和底层数据布局。

在模型部署过程中，RKNN 模型的输入输出结构需要结合实际打印信息进行分析。YOLO 模型在 RKNN 中可能输出多个尺度张量，后处理不能简单套用通用代码，而要根据输出维度、量化参数、类别数量和坐标变换方式进行适配。项目中曾出现检测框位置异常、目标数量不稳定、置信度阈值不合适等问题，最终通过重新分析模型输出、多尺度解码、NMS 策略和坐标还原过程逐步改善检测效果。

多目标跟踪阶段接入 ByteTrack 后，系统从单帧检测升级为跨帧目标管理。`track_id` 的引入使同一目标的连续行为可以被记录，为长时间停留、区域闯入和轨迹分析提供了基础。但实际测试也发现，检测框抖动、遮挡、小目标漏检和短时丢帧都会导致 ID 切换，因此跟踪稳定性不仅依赖跟踪算法，也依赖检测质量、摄像头视角和后处理参数。

在系统展示方面，项目从本地调试逐步扩展到 RTSP 推流、MediaMTX 转发、FastAPI 接口和 Web 后台展示。这个过程体现了工程系统中“算法结果”和“用户界面”分离的重要性。视频流适合展示直观画面，JSON 数据适合展示统计指标和事件记录，两者结合后才能形成完整的智能监控系统。总体来看，本作品不仅完成了一个可运行的边缘 AI 视频分析系统，也让我对嵌入式视觉系统的采集链路、推理链路、通信链路和应用展示链路形成了完整认识。

## 第五部分 参考文献

---

- [1] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLOv8 Documentation[EB/OL].  
<https://docs.ultralytics.com>.
- [2] Zhang Y, Sun P, Jiang Y, et al. ByteTrack: Multi-Object Tracking by Associating Every Detection Box[C]. European Conference on Computer Vision, 2022.
- [3] Bewley A, Ge Z, Ott L, et al. Simple Online and Realtime Tracking[C]. IEEE International Conference on Image Processing, 2016.
- [4] Rockchip. RKNN Toolkit2 User Guide[EB/OL]. Rockchip Developer Documentation.
- [5] Rockchip. RGA Developer Guide[EB/OL]. Rockchip Developer Documentation.
- [6] Linux Media Infrastructure. Video4Linux2 API Specification[EB/OL].  
<https://www.kernel.org/doc/html/latest/userspace-api/media/v4l/v4l2.html>.
- [7] GStreamer Project. GStreamer Application Development Manual[EB/OL].  
<https://gstreamer.freedesktop.org/documentation/>.
- [8] MediaMTX Project. MediaMTX Documentation[EB/OL]. <https://github.com/bluenviron/mediamtx>.
- [9] OpenCV Team. OpenCV Documentation[EB/OL]. <https://docs.opencv.org>.
- [10] FastAPI. FastAPI Documentation[EB/OL]. <https://fastapi.tiangolo.com>.